

## **Ciência da Computação**

---

### **Teoria da Computação**

Prof. Giovanni Ely Rocco (gerocco@ucs.br)



# Computabilidad

# Computabilidade

O que é uma solução computável?

Existem problemas sem solução computacional?

Quais são os limites do que pode ser computado?

**Algoritmo**

*método efetivo para  
decidir validade de  
dada expressão lógica*

Abu Ja'far Mohammed ibn Mûsâ **al-Khowârizm**, 825

Algoritmo de Euclides, 300 aC

*Cálculo Lambda  
(Church, 1936)*

*Funções Recursivas  
(Kleene, 1938)*

**Tese de Church-Turing**

**Máquina Computacional**  
computabilidade efetiva

*Entscheidungsproblem* (Hilbert, 1928)

**Alan Turing, 1936**

# Computabilidade

**Programas**

Equivalências de Máquinas

**Máquina de Turing**

**Máquinas Universais**

**Máquina Norma, 1976**

**Classes de  
Solucionabilidade  
de Problemas**

Máquina de Post, 1936

Hierarquia de

**Classes de Máquinas**

problemas solucionáveis

Máquinas com Pilhas

**Problemas de Decisão**

Não determinismo

problemas não-solucionáveis

problema geral de verificação de software

problemas **computáveis**

Princípio da Redução

**Problema da Parada**

problemas **não-computáveis**

---

# Programas, Máquinas e Computações

## Programas

Conjunto estruturado de instruções que capacitam uma máquina a aplicar sucessivamente certas operações básicas e testes sobre os dados iniciais fornecidos, com o objetivo de transformar estes dados numa forma desejável.

## Máquinas

Modelos que têm como objetivo suprir todas as informações necessárias para que a computação de um programa possa ser descrita, dando significado (semântica) aos identificadores das operações e testes.

## Computações

Histórico de funcionamento da máquina para o programa, considerando um valor e uma configuração inicial.

# Programas

---

Estruturação **Monolítica**

Estruturação **Iterativa**

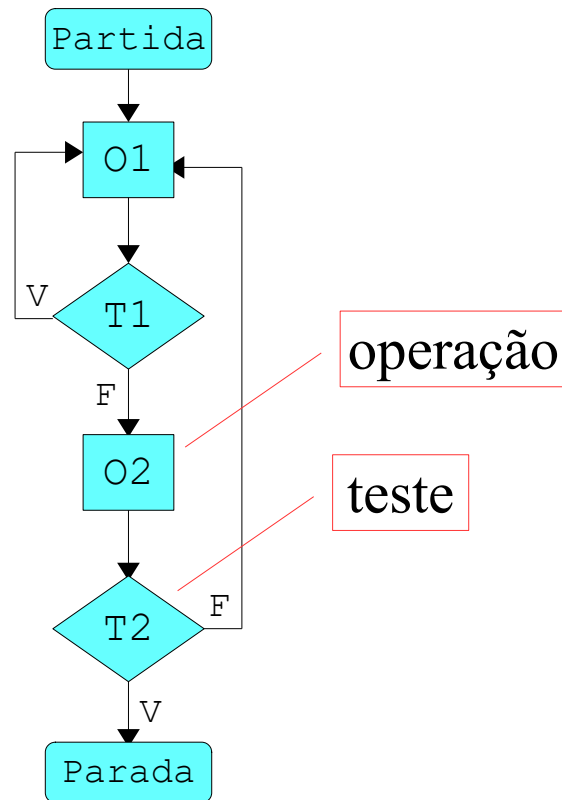
Estruturação **Recursiva**

Composição **Sequencial**

Composição **Não-determinística**

Composição **Concorrente**

# Programas



## Estruturação Monolítica

A lógica é distribuída por todo o bloco (monólito) que constitui o programa, estruturado usando desvios condicionais e incondicionais.

$$P = (I, r)$$

I: Conjunto de Instruções Rotuladas

r: Rótulo Inicial

```
r1: faça O1 vá_para r2
r2: se T1 então vá r1 senão vá r3
r3: faça O2 vá_para r4
r4: se T2 então vá r5 senão vá r1
r5:
```



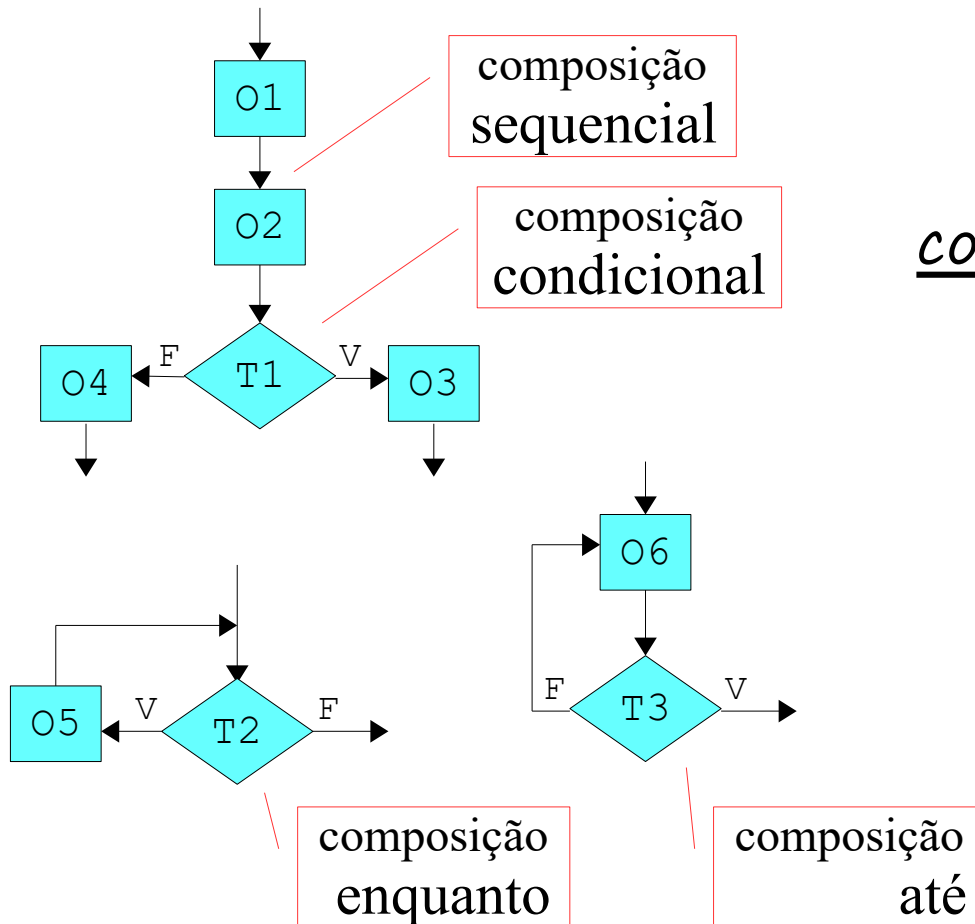
# Programas

## Estruturação Iterativa

Programa com estruturas de controle de ciclos ou repetições  
(para melhor estruturação dos desvios).

$P = P_1; P_2; P_3; \dots; P_n$

$P_{1..n}$ : Programas Iterativos



O1; O2

se T1 então (O3) senão (O4)

enquanto T2 faça (O5)

faça (O6) até T3

# Programas

## Estruturação Recursiva

*Recursão com sub-rotinas é uma forma indutiva de definir programas.*

Programa com estruturação hierárquica, com sub-rotinas e recursão, que possibilitam níveis diferenciados de abstração.

Exemplo:

**P é R;S onde**  
**R** def 01; se T1 então R senão T  
**T** def 03; 04; se T3 então S senão √  
**S** def se T2 então √ senão 02; R

**P é E<sub>0</sub> onde R<sub>1</sub> def E<sub>1</sub>, R<sub>2</sub> def E<sub>2</sub>, ..., R<sub>n</sub> def E<sub>n</sub>**

E<sub>0</sub>: Expressão Inicial

E<sub>k</sub>: Expressão que define Sub-rotina R<sub>k</sub>

# Exercícios

## Perguntas:

1. Considerando uma composição de repetição com teste enquanto condição for verdadeira no final (tal qual o funcionamento do comando “do-while” em C)...
2. Considerando uma composição de repetição “para” (tal qual a composição do comando “for” em C)...

como é sua representação em fluxograma?

como especificar com uma estruturação iterativa?

como especificar com uma estruturação monolítica?

como especificar com uma estruturação recursiva?

# Exercícios

```
do { comando;  
  } while (condição);
```

## Iterativo:

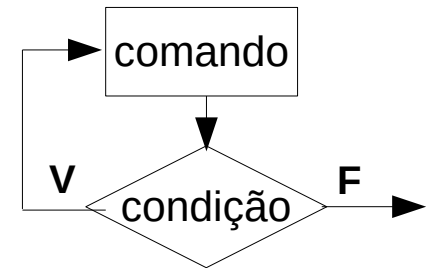
```
comando;  
enquanto condição faça comando
```

## Monolítico:

```
1: faça comando vá_para 2  
2: se condição então vá_para 1 senão vá_para 3
```

## Recursivo:

```
DoWhile é F onde  
    F def comando; se condição então F senão √
```



# Exercícios

```
for (inicial; condição; passo)  
    comando;
```

## Iterativo:

```
inicial;  
enquanto condição faça (comando; passo)
```

## Monolítico:

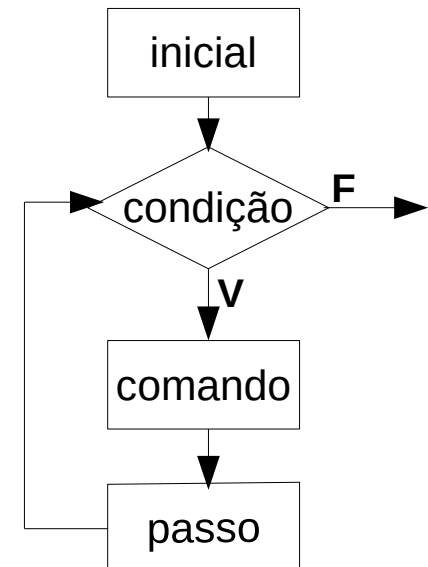
```
1: faça inicial vá_para 2  
2: se condição então vá_para 3 senão vá_para 5  
3: faça comando vá_para 4  
4: faça passo vá_para 2
```

## Recursivo:

FOR é I onde

I def inicial; F

F def se condição então (comando; passo; F) senão √



# Exercícios

e considerando uma adequação da repetição para otimização do algoritmo...

## Recursivo:

REPETIÇÃO é I onde

I def inicial; F

F def se condição então G senão √

G def comando;

se condição então (passo; F)  
senão √

## Monolítico:

1: faça inicial vá\_para 2

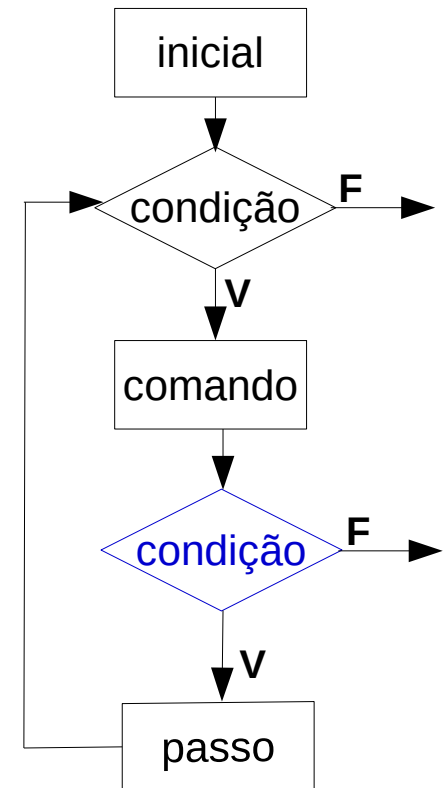
2: se condição então vá\_para 3 senão vá\_para 6

3: faça comando vá\_para 4

4: se condição então vá\_para 5 senão vá\_para 6

5: faça passo vá\_para 2

## Iterativo: ???



*Programa é simplesmente uma sequência de símbolos,  
sem significado semântico.*

*A natureza das operações e dos testes  
é especificada na definição de máquina, que permite  
dar uma interpretação semântica do programa.*

## **Máquinas**    **$M = (V, X, Y, \pi_X, \pi_Y, \Pi_F, \Pi_T)$**

---

**V:** Conjunto de Valores de Memória

**X:** Conjunto de Valores de Entrada

**Y:** Conjunto de Valores de Saída

**$\pi_X$ :** Função de Entrada tal que:  **$\pi_X: X \rightarrow V$**

**$\pi_Y$ :** Função de Saída tal que:  **$\pi_Y: V \rightarrow Y$**

**$\Pi_F$ :** Conjunto de Interpretações de Operações tal que  
para cada identificador de operação F interpretada por M  
existe uma única função:  **$\pi_F: V \rightarrow V$  em  $\Pi_F$**

**$\Pi_T$ :** Conjunto de Interpretações de Testes tal que  
para cada identificador de teste T interpretada por M  
existe uma única função:  **$\pi_T: V \rightarrow \{\text{verdadeiro}, \text{falso}\}$  em  $\Pi_T$**

## Uma Máquina de Dois Registradores:

$DoisReg = (\mathbb{N}^2, \mathbb{N}, \mathbb{N}, Arm\_A, Ret\_B, \{Sub\_A, Add\_B\}, \{Zero\_A\})$

$\mathbb{N}^2$ : conjunto de valores de memória

$\mathbb{N}$ : conjunto de valores de entrada e de saída

$Arm\_A: \mathbb{N} \rightarrow \mathbb{N}^2$  [função de entrada]

$\forall n \in \mathbb{N}: Arm\_A(n) = (n, 0)$

$Ret\_B: \mathbb{N}^2 \rightarrow \mathbb{N}$  [função de saída]

$\forall (n, m) \in \mathbb{N}^2: Ret\_B(n, m) = m$

$Sub\_A: \mathbb{N}^2 \rightarrow \mathbb{N}^2$  [operação]

$\forall (n, m) \in \mathbb{N}^2: Sub\_A(n, m) = (n-1, m)$  se  $n \neq 0$

$Sub\_A(n, m) = (0, m)$  se  $n = 0$

$Add\_B: \mathbb{N}^2 \rightarrow \mathbb{N}^2$  [operação]

$\forall (n, m) \in \mathbb{N}^2: Add\_B(n, m) = (n, m+1)$

$Zero\_A: \mathbb{N}^2 \rightarrow \{\text{verdadeiro}, \text{falso}\}$  [teste]

$\forall (n, m) \in \mathbb{N}^2: Zero\_A(n, m) = V$  se  $n = 0$ ;  $Zero\_A(n, m) = F$  se  $n \neq 0$



Sendo  $M = (V, X, Y, \pi_X, \pi_Y, \Pi_F, \Pi_T)$  e  $P = (l, r)$ ,  
com  $L$  o correspondente conjunto de rótulos...

## Computações

---

Uma *Computação* do Programa  $P$  na Máquina  $M$  é  
uma cadeia (finita ou infinita) de pares  $L \times V$ :  
 $(s_0, v_0) (s_1, v_1) (s_2, v_2) \dots$

onde:

$(s_0, v_0)$  é tal que  $s_0 = r$  é o rótulo inicial do programa  $P$   
e  $v_0$  é o valor de memória na máquina  $M$

e cada par  $(s_k, v_k)$  tem-se que:

se  $s_k$ : *faça F vá\_para r'* [rótulo de operação]

então  $(s_{k+1}, v_{k+1}) = (r', \pi_F(v_k))$  é par subsequente de  $(s_k, v_k)$

se  $s_k$ : *faça v vá\_para r'* [rótulo de operação]

então  $(s_{k+1}, v_{k+1}) = (r', v_k)$  é par subsequente de  $(s_k, v_k)$

se  $s_k$ : *se T então vá\_para r' senão vá\_para r''* [rótulo de teste]

então  $(s_{k+1}, v_{k+1})$  é par subsequente de  $(s_k, v_k)$ , com  $v_{k+1} = v_k$

e  $s_{k+1} = r'$ , se  $\pi_T(v_k) = \text{verdadeiro}$

$s_{k+1} = r''$ , se  $\pi_T(v_k) = \text{falso}$

# Computações

## Considerações

- \* *Computação Finita: se a cadeia que a define é finita.*
- \* *Computação Infinita: se a cadeia que a define é infinita.*
- \* *Em uma computação finita,  
a última configuração tem um rótulo final.*
- \* *Em uma computação infinita,  
rótulo algum da cadeia é final (não há parada).*
- \* *Um teste não altera o valor corrente da memória.*
- \* *Para um dado valor inicial de memória,  
a correspondente cadeia de computações é única, ou seja,  
a computação é determinística (produz sempre a mesma saída).*

# Computações

em Programas Recursivos

- \* *Computação Finita: se a cadeia que a define é finita.*
- \* *Computação Infinita: se a cadeia que a define é infinita.*
- \* *Em uma computação finita,  
a última configuração apenas resta a expressão  $v$ .*
- \* *Em uma computação infinita,  
na cadeia de computações não há expressão alguma  $v$ .*
- \* *Um teste não altera o valor corrente da memória,  
assim como não altera uma referência à sub-rotina.*
- \* *Para um dado valor inicial de memória,  
a correspondente cadeia de computações é única, ou seja,  
a computação é determinística (produz sempre a mesma saída).*

A computação de um programa deve estar associada a uma entrada e uma saída, e espera-se que a resposta (saída) seja gerada em um tempo finito.

## Função Computada

---

Por um programa monolítico sobre uma máquina:

- \* a computação inicia na instrução identificada pelo rótulo inicial com a memória contendo o valor inicial resultante da aplicação da função de entrada sobre o dado fornecido;
- \* executa, passo a passo, testes e operações na ordem determinada pelo programa, até atingir um rótulo final, quando para;
- \* o valor da função computada pelo programa é o valor resultante da aplicação da função de saída ao valor da memória quando da parada.

A computação de um programa deve estar associada a uma entrada e uma saída, e espera-se que a resposta (saída) seja gerada em um tempo finito.

## Função Computada

---

Por um programa recursivo sobre uma máquina:

- \* a computação inicia na expressão inicial com a memória contendo o valor inicial resultante da aplicação da função de entrada sobre o dado fornecido;
- \* executa, passo a passo, testes e operações na ordem determinada pelo programa, até a expressão de sub-rotina resultante seja a expressão vazia, quando para;
- \* o valor da função computada pelo programa é o valor resultante da aplicação da função de saída ao valor da memória quando da parada.

**Rótulo inicial ou  
Expressão inicial**

Função de entrada:  
 $\pi_x(V_n)$

$(r, v)$

...

...

$(r', v')$

Computações:  
 $\Pi_F: \pi_f(V_n) \rightarrow V_n$   
 $\Pi_T: \pi_t(V_n) \rightarrow V \text{ ou } F$

**Rótulo final ou  
Expressão vazia**

Função de saída:  
 $\pi_y(V_n)$

## Função Computada

$\langle P, M \rangle: X \rightarrow Y$

$\langle P, M \rangle(x) = \pi_Y(V_n)$

# Exemplos

*Considerando uma máquina de dois registradores*

*DoisRegs = ( $\mathbb{N}^2, \mathbb{N}, \mathbb{N}, \text{Arm\_A}, \text{Ret\_B}, \{\text{Sub\_A}, \text{Add\_B}\}, \{\text{Zero\_A}\})$*

*Programa para somar valores ( $B = A+B$ )?*

*Programa Iterativo*

se Zero\_A então ✓  
senão faça (Sub\_A; Add\_B) até Zero\_A

*Programa Recursivo*

Soma é R onde  
R def se Zero\_A então ✓ senão (S; R)  
S def Sub\_A; Add\_B

# Exemplos

*Considerando uma máquina de dois registradores*

$\text{DoisRegs} = (\mathbb{N}^2, \mathbb{N}, \mathbb{N}, \text{Arm\_A}, \text{Ret\_B}, \{\text{Sub\_A}, \text{Add\_B}\}, \{\text{Zero\_A}\})$

*e um programa monolítico para somar valores:*

```
1: se Zero_A então vá_para 9 senão vá_para 2
2: faça Sub_A vá_para 3
3: faça Add_B vá_para 1
```

*Computações para entrada 2 em A e 1 em B?*

|               |                                                       |
|---------------|-------------------------------------------------------|
| $(1, (2, 1))$ | entrada do valor 2 em A e do valor 1 em B             |
| $(2, (2, 1))$ | em 1, como $A \neq 0$ desviou para 2                  |
| $(3, (1, 1))$ | em 2, subtraiu do registrador A e desviou para 3      |
| $(1, (1, 2))$ | em 3, adicionou ao registrador B e desviou para 1     |
| $(2, (1, 2))$ | ...                                                   |
| $(3, (0, 2))$ | ...                                                   |
| $(1, (0, 3))$ | ...                                                   |
| $(9, (0, 3))$ | em 1, como $A = 0$ desviou para 9 (computação finita) |



# Exemplos

Considerando a máquina de dois registradores e o programa para somar dois valores (*ProgSomador*)...

Função Computada para entrada 2 em A e 1 em B?

$$\langle \text{ProgSomador}, \text{DoisRegs} \rangle(2,1) = 3$$

Computações e Função Computada?

Computação é a sequência de instruções executadas para um programa e os diferentes valores da memória ao longo de sua execução (*histórico do funcionamento da máquina para o programa*).

Função computada é o resultado obtido ao fim de uma computação, desde que a computação seja finita.

# Exercícios

Considerando uma máquina de um registrador

$$UmReg = (\mathbb{N}, \mathbb{N}, \mathbb{N}, ID_A, ID_A, \{ADD, SUB\}, \{ZERO\})$$

Definição formal da máquina?

Programa monolítico para duplicar valor?

E com uma estruturação recursiva é possível?

► Elaborar um programa com estruturação recursiva e apresentar as computações do programa na máquina para um valor de entrada 2.

# Exercícios

Programa *Duplica* para máquina *UmReg*:

$UmReg = (IN, IN, IN, ID_A, ID_A, \{ADD, SUB\}, \{ZERO\})$

**Duplica é R onde**

**R def se ZERO então  $\sqrt{\phantom{x}}$  senão (SUB;R;ADD;ADD)**

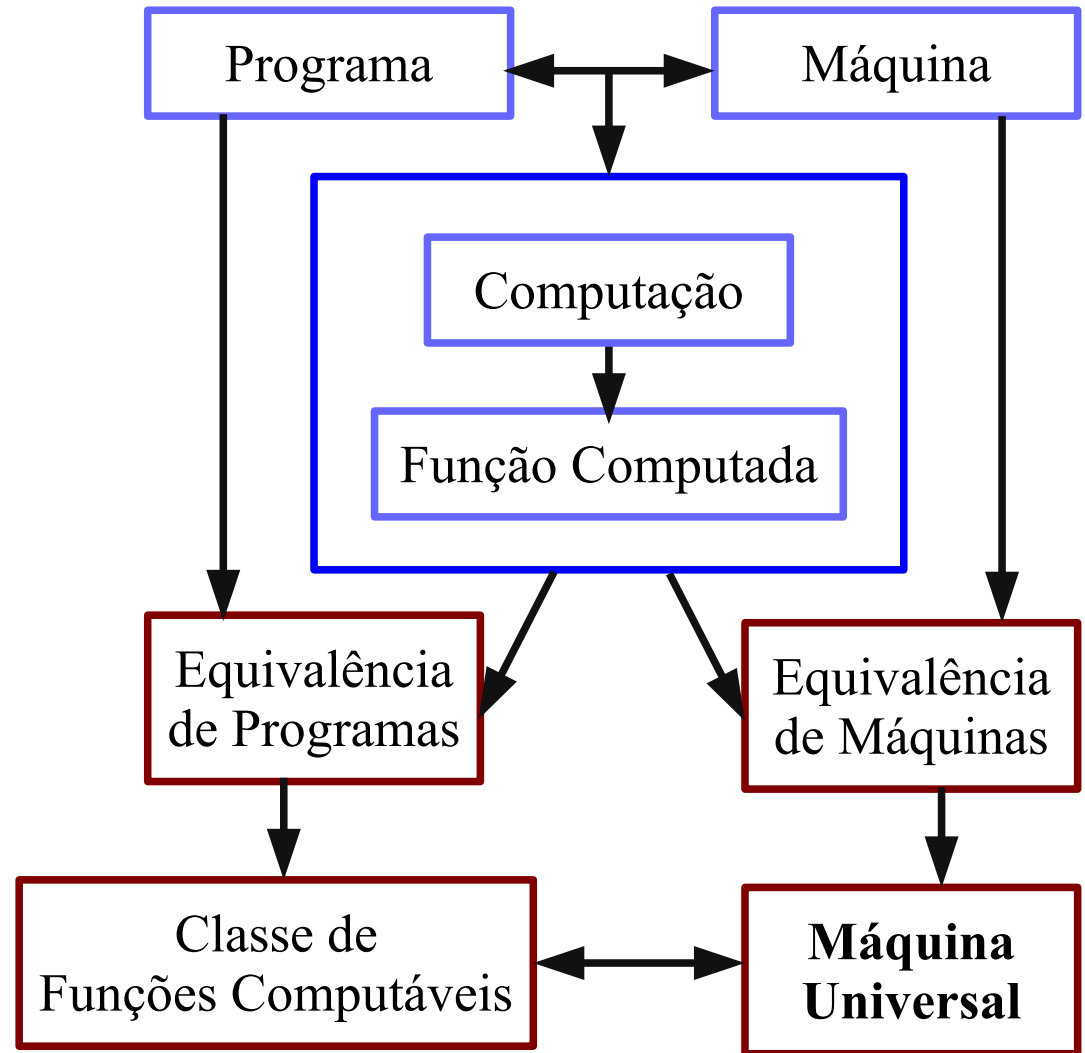
Computações do programa  
para valor de entrada 2:

(R, 2)  
( (SUB;R;ADD;ADD) , 2)  
( (R;ADD;ADD) , 1)  
( ( (SUB;R;ADD;ADD) ;ADD;ADD) , 1)  
( ( (R;ADD;ADD) ;ADD;ADD) , 0)  
( ( (  $\sqrt{\phantom{x}}$  ) ;ADD;ADD) ;ADD;ADD) , 0)  
( ( (ADD;ADD) ;ADD;ADD) , 0)  
( ( (ADD) ;ADD;ADD) , 1)  
( (ADD;ADD) , 2)  
( (ADD) , 3)  
(  $\sqrt{\phantom{x}}$  , 4)

Função Computada:

**$\langle Duplica, UmReg \rangle (2) = 4$**





## Relações Entre Conceitos